

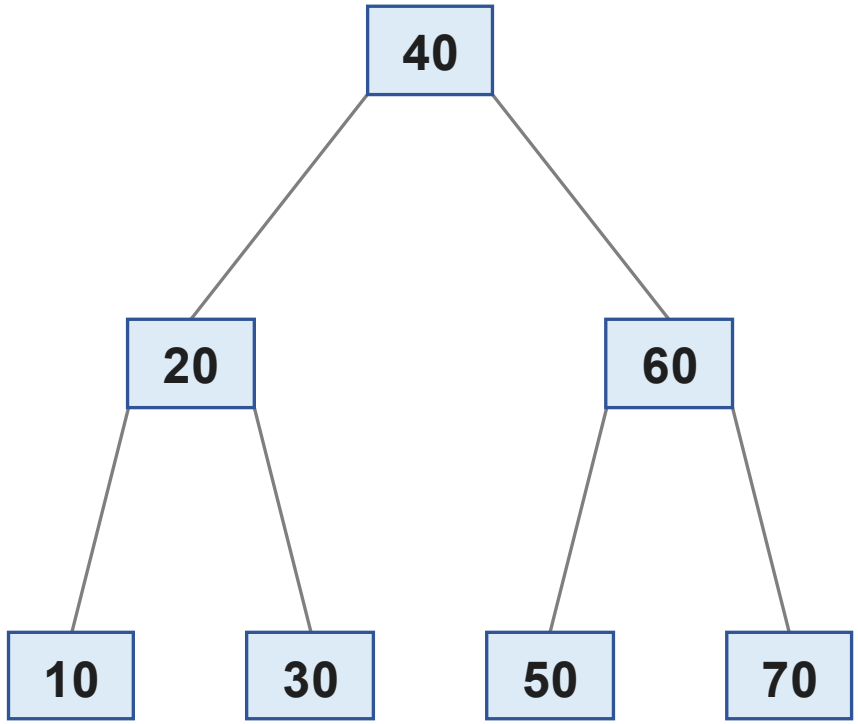
భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

B Trees

B+ Trees

Hashing Applications

Recap: Balanced Binary Search Trees



- AVL / red-black trees: height = **O($\log_2$ n)**
- Each node: **1 key, 2 children**
- Fine when everything is in **main memory**
- But what if n = 10^9 records on **disk**?

The Memory Hierarchy

CPU registers / cache · ~1 ns

Main memory (RAM) · ~50–100 ns

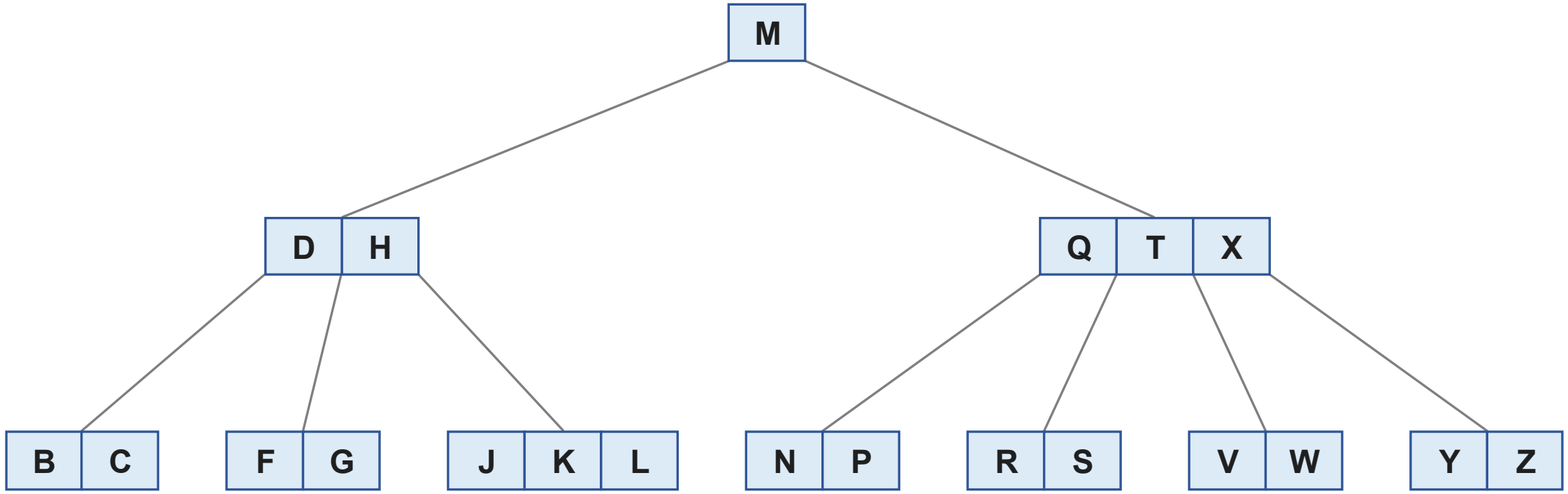
Disk (SSD / HDD) · ~0.1–10 ms

- A disk access is about **10^5 times slower** than a memory access
- Disk reads whole **pages (blocks)** of 4–16 KB at a time

Key Idea: Count Disk Accesses

- Running time is dominated by **DISK-READ** and **DISK-WRITE** of pages
- One node = one page → reading a node costs **one disk access**
- So: minimise the number of nodes visited = the **height** of the tree
- Make nodes **fat**: hundreds of keys per node

A B-Tree at a Glance



- A node with **n keys** has **n + 1 children**
- Keys inside a node are **sorted**; they split the key range for the children
- All leaves are at the **same depth**

Definition

- Each node x stores **x.n** keys in sorted order: $x.key_1 \le x.key_2 \le \ldots \le x.key_n$
- x.**leaf** = TRUE if x is a leaf
- An internal node has **x.n + 1** children; keys separate their ranges:
  - $k_1 \le x.key_1 \le k_2 \le x.key_2 \le \ldots \le x.key_n \le k_{n+1}$
- All leaves have the **same depth** h (the height)
- Node sizes are bounded by the **minimum degree t ≥ 2**

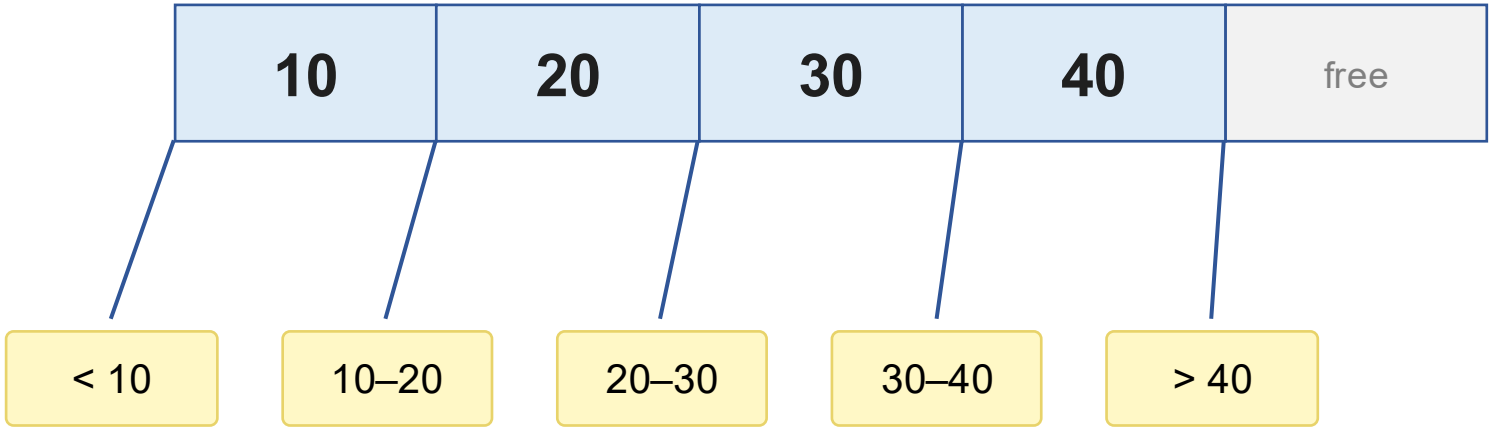
Minimum Degree t

Node	Keys	Children
Root (non-empty)	1 … 2t − 1	2 … 2t (if internal)
Other internal	t − 1 … 2t − 1	t … 2t
Leaf	t − 1 … 2t − 1	0

- A node with **2t − 1** keys is **full**
- t = 2 → every internal node has 2, 3 or 4 children: the **2-3-4 tree**
- In practice t is large (e.g., 50–2000) so a node fills a disk page

A Node with t = 3

n = 4 keys (max 2t − 1 = 5), so 5 child pointers



- Child c_i holds the keys between key_{i-1} and key_i

The Node in C

```c
#define T 3                         /* minimum degree (t >= 2)        */
#define MAX_KEYS (2 * T - 1)        /* a full node has 2t-1 keys      */

typedef struct BTreeNode {
    int  n;                         /* number of keys stored          */
    int  keys[MAX_KEYS];            /* keys in increasing order       */
    struct BTreeNode *c[2 * T];     /* child pointers (n+1 used)      */
    bool leaf;                      /* true if node is a leaf         */
} BTreeNode;

BTreeNode *allocate_node(bool leaf) {
    BTreeNode *x = malloc(sizeof(BTreeNode));
    x->n = 0;
    x->leaf = leaf;
    for (int i = 0; i < 2 * T; i++) x->c[i] = NULL;
    return x;
}
```

Notes

- **T**: minimum degree
- Arrays sized for a **full** node
- **leaf** flag avoids checking children
- On disk: one node = one page

How Tall Can a B-Tree Be?

Depth	Min. nodes	Min. keys
0	1	1
1	2	$2(t - 1)$
2	$2t$	$2t(t - 1)$
h	$2t^{h-1}$	$2t^{h-1}(t - 1)$

- Root ≥ 1 key; every other node ≥ $t - 1$ keys and ≥ t children
- Add the levels: $n \geq 1 + 2(t-1)\cdot(t^0 + t^1 + \ldots + t^{h-1}) = 2t^h - 1$

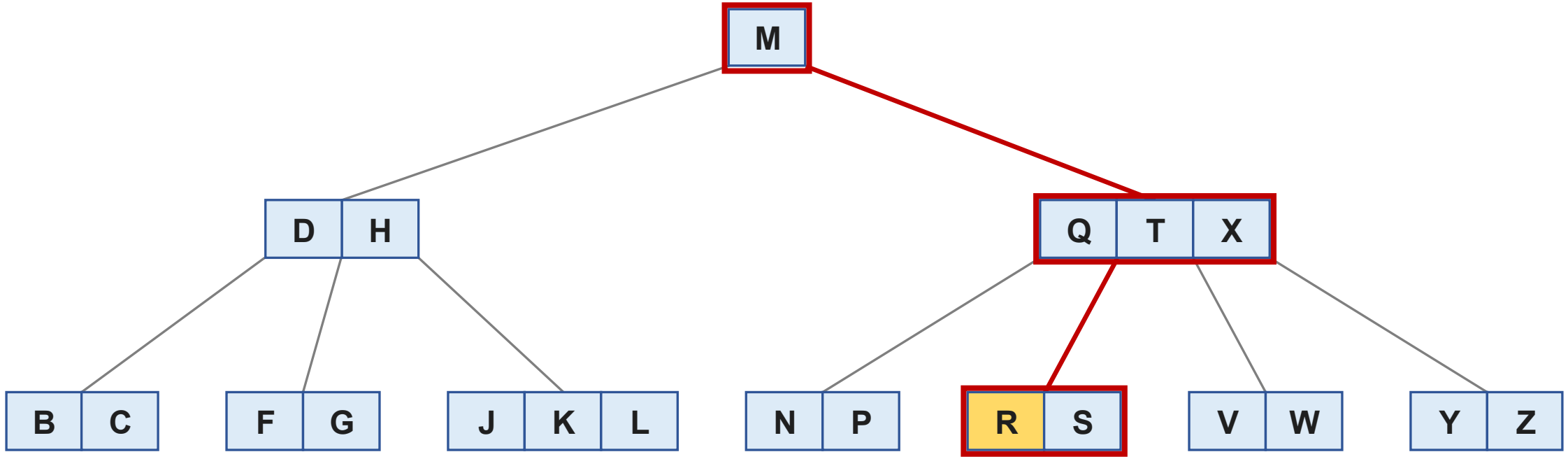
Theorem 18.1: $h \leq \log_t ((n + 1) / 2)$

Height in Numbers

n (keys)	Binary ($\log_2 n$)	B-tree t = 100	B-tree t = 1000
10^6	≈ 20	≤ 2	≤ 1
10^9	≈ 30	≤ 4	≤ 2
10^{12}	≈ 40	≤ 5	≤ 3

- Heights from h ≤ log_t((n+1)/2), rounded down
- Larger t → **fewer disk reads** (but more CPU work per node)

Search: Generalising BST Search



- Search for **R**: in each node, scan for the first key ≥ R
- Found? stop. Else descend into the matching child (one **DISK-READ**)
- Visits ≤ h + 1 nodes

Search: The Code

```
/* ---------------- SEARCH ---------------- */
BTreeNode *btree_search(BTreeNode *x, int k, int *idx) {
    int i = 0;
    while (i < x->n && k > x->keys[i])      /* find first key >= k */
        i++;
    if (i < x->n && k == x->keys[i]) {      /* found in this node  */
        *idx = i;
        return x;
    }
    if (x->leaf)                            /* nowhere to go       */
        return NULL;
    return btree_search(x->c[i], k, idx);   /* DISK-READ(x->c[i])  */
}
```

Cost

- Disk accesses: **O(h) = O(log_t n)**
- CPU time: **O(t · log_t n)**
- Binary search inside a node → O(log t) per node

Creating an Empty Tree

```
/* B-TREE-CREATE: an empty tree is a single empty leaf */
BTreeNode *btree_create(void) { return allocate_node(true); }
```

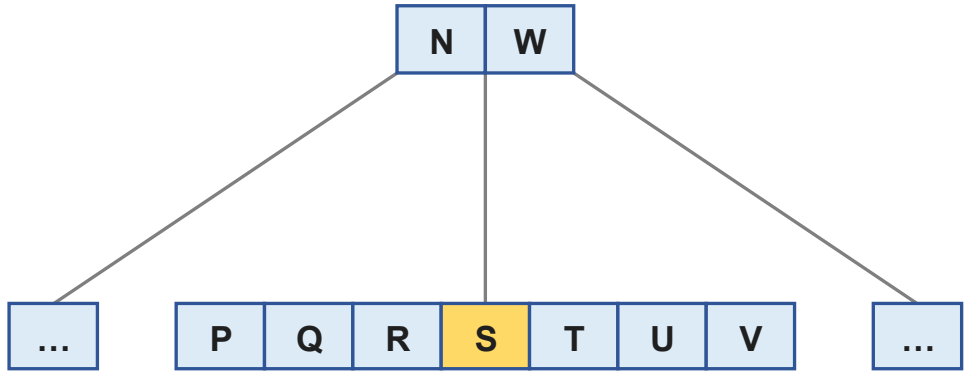
- A new tree is a single, empty **leaf** root
- O(1) disk operations
- All further nodes are created by **splitting**

Insertion: The Challenge

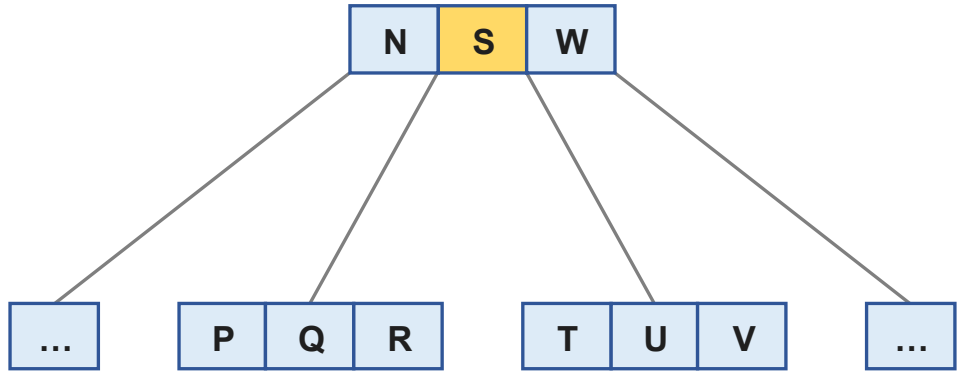
- New keys always go into a **leaf** (like a BST)
- But a leaf may be **full** ($2t - 1$ keys) — we cannot add a key
- Fix: **split** the full node around its **median** key; the median moves **up** to the parent
- What if the parent is also full?
  - Split that too

Splitting a Full Child (t = 4)

Before: child y = $x.c_2$ is full (7 keys)



After: median S moves up

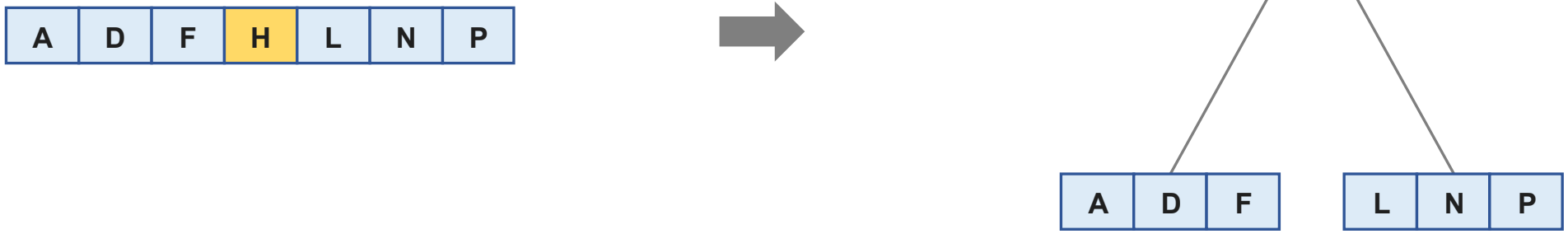


- y keeps the lower **t − 1** keys, new node z gets the upper **t − 1** keys
- Median key goes up into the parent x; the tree **does not** get taller

Split Child: The Code

```
/* ---------------- INSERT ---------------- */
/* Split the full child y = x->c[i] around its median key. */
void btree_split_child(BTreeNode *x, int i) {
    BTreeNode *y = x->c[i];
    BTreeNode *z = allocate_node(y->leaf);
    z->n = T - 1;
    for (int j = 0; j < T - 1; j++)         /* upper t-1 keys -> z  */
        z->keys[j] = y->keys[j + T];
    if (!y->leaf)
        for (int j = 0; j < T; j++)         /* upper t children -> z */
            z->c[j] = y->c[j + T];
    y->n = T - 1;                           /* y keeps lower t-1 keys */
    for (int j = x->n; j > i; j--)          /* make room for z in x  */
        x->c[j + 1] = x->c[j];
    x->c[i + 1] = z;
    for (int j = x->n - 1; j >= i; j--)     /* make room for median  */
        x->keys[j + 1] = x->keys[j];
    x->keys[i] = y->keys[T - 1];            /* median moves up       */
    x->n++;
}
```

Splitting the Root: How a B-Tree Grows



- If the **root** is full: create a new empty root s, make the old root its child, split it
- This is the **only** way the height increases — at the **top**, not the bottom
- That is why all leaves stay at the same depth

Insert: The Code

```
/* Returns the (possibly new) root. The tree grows only at the root. */
BTreeNode *btree_insert(BTreeNode *root, int k) {
    if (root->n == MAX_KEYS) {              /* root full: split it      */
        BTreeNode *s = allocate_node(false);
        s->c[0] = root;
        btree_split_child(s, 0);
        btree_insert_nonfull(s, k);
        return s;                           /* height grows by one      */
    }
    btree_insert_nonfull(root, k);
    return root;
}
```

- Returns the new root when the root was split

Insert into a Non-Full Node

```
/* Insert k into node x, which is guaranteed NOT to be full. */
void btree_insert_nonfull(BTreeNode *x, int k) {
    int i = x->n - 1;
    if (x->leaf) {
        while (i >= 0 && k < x->keys[i]) { /* shift larger keys right */
            x->keys[i + 1] = x->keys[i];
            i--;
        }
        x->keys[i + 1] = k;
        x->n++;
    } else {
        while (i >= 0 && k < x->keys[i])
            i--;
        i++;                                /* child to descend into   */
        if (x->c[i]->n == MAX_KEYS) {       /* full? split it first    */
            btree_split_child(x, i);
            if (k > x->keys[i])
                i++;
        }
        btree_insert_nonfull(x->c[i], k);
    }
}
```

Invariant

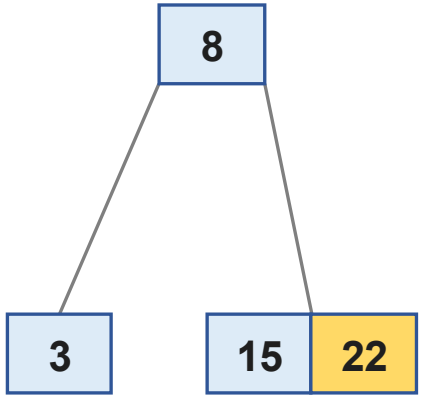
- **Leaf**: shift and insert (like insertion sort)
- **Internal**: find the child; if it is full, **split first**
- After the split, pick the correct half
- Recursion always enters a **non-full** node

Worked Example (t = 2): Insert 8, 15, 3, 22

After 8, 15, 3: root is full (3 keys)



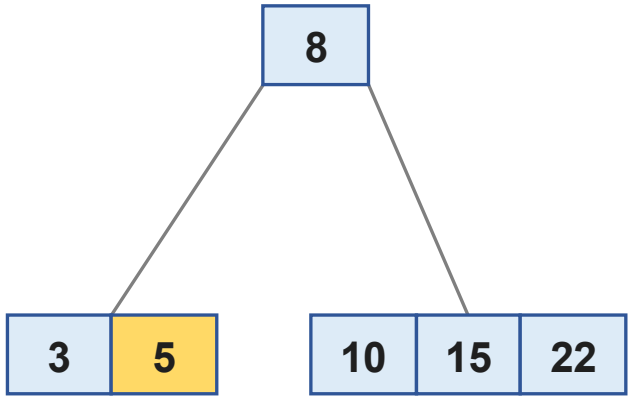
Insert 22: split root first, then insert



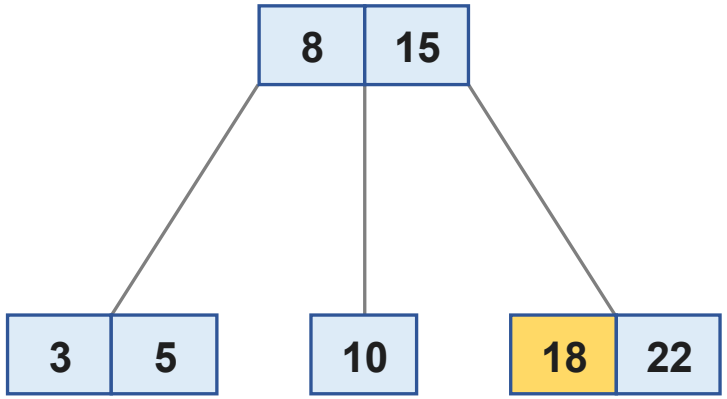
- t = 2: at most **3 keys** per node (a 2-3-4 tree)
- Height grows from 0 to 1

Worked Example: Insert 10, 5, then 18

After 10 and 5: leaf [10 15 22] is full

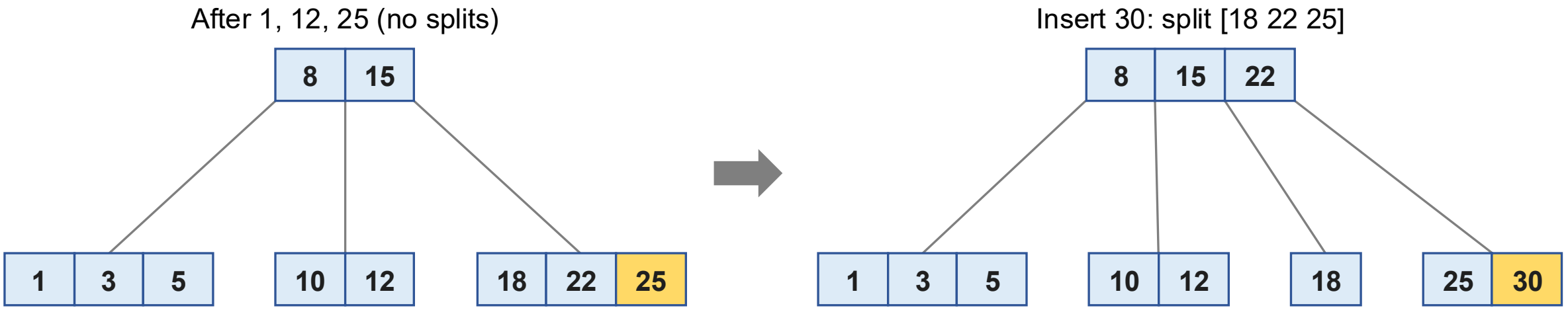


Insert 18: split [10 15 22] on the way down



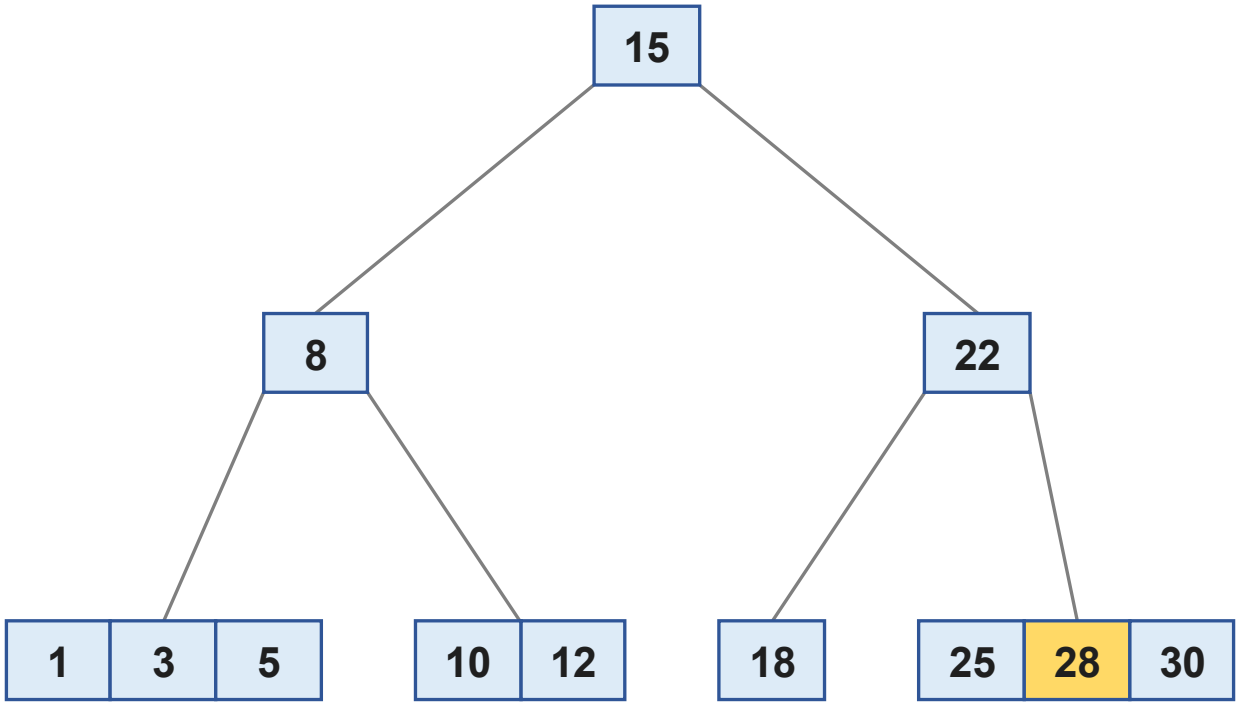
- Median **15** moves up into the root
- 18 then goes into the right half [18 22]

Worked Example: Insert 1, 12, 25, 30



- Median **22** moves up — now the **root is full**

Worked Example: Insert 28



- Root [8 15 22] is full → split it: **15** becomes the new root
- Then 28 descends and lands in [25 28 30]

Sample C Program

```c
int main(void) {
    BTreeNode *root = btree_create();
    int keys[] = {10, 20, 5, 6, 12, 30, 7, 17, 3, 4, 2, 50, 60, 1};
    for (int i = 0; i < (int)(sizeof keys / sizeof keys[0]); i++)
        root = btree_insert(root, keys[i]);
    printf("After insertions:\n");
    btree_print(root, 0);
```

```
After insertions:
[5 10 20]
    [1 2 3 4]
    [6 7]
    [12 17]
    [30 50 60]
search(17): found
```

- Full source (with deletion and a test driver): **btree.c**

Deletion: Strategy

- Keys can be deleted from **any** node, not just leaves
- Danger: a node may drop below **$t - 1$** keys
- Idea: before descending into a child, make sure it has **at least t keys**
  - then one key can always be removed without backing up
- Only the **root** may end up empty → the tree **shrinks** at the top

Motivation: Range Queries

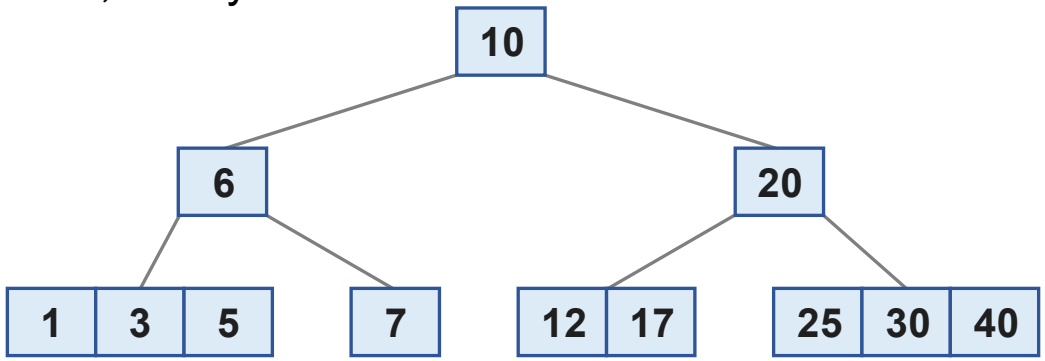
- Databases often ask: **“all students with marks between 60 and 80”**
- In a B-tree, keys are spread over **all levels** → a range scan keeps going up and down
- Idea: keep **all keys in the leaves**, chain the leaves left-to-right
- Internal nodes only **route** the search → more keys fit per page → even shorter tree

B+ Tree: Definition

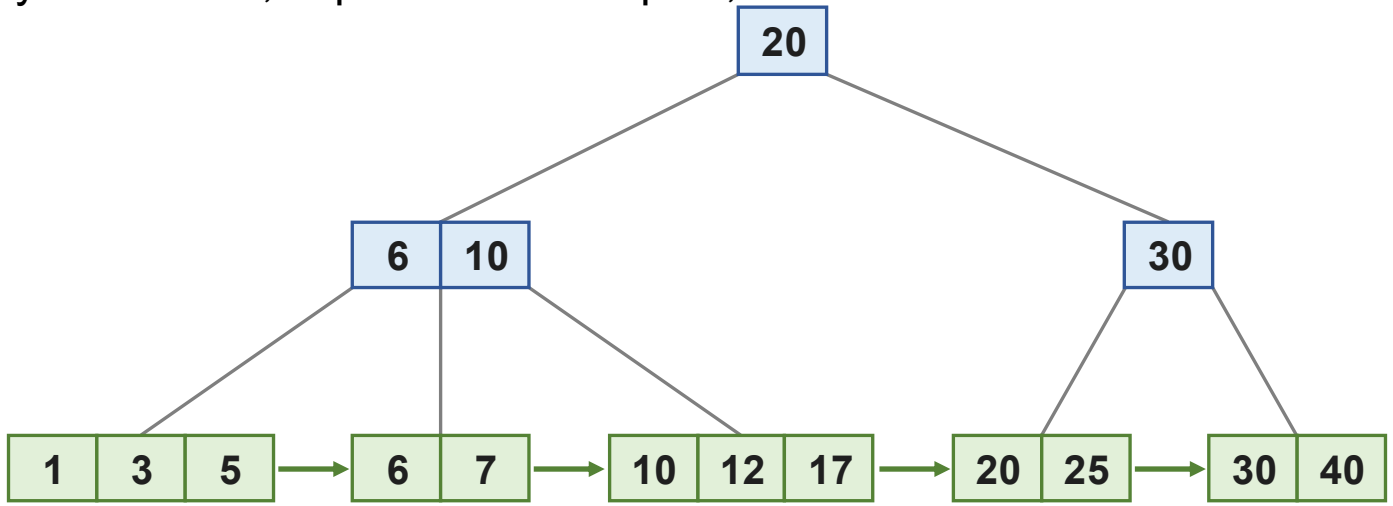
- All keys (and records / record pointers) are stored in the **leaves**
- Internal nodes store **separator keys** — copies that guide the search
- Leaves are linked: each leaf has a **next** pointer
- All leaves at the same depth; nodes at least **half full**
  - Order M = 4 here: internal nodes 2–4 children, leaves 2–3 keys

Same Keys: B-Tree vs. B+ Tree

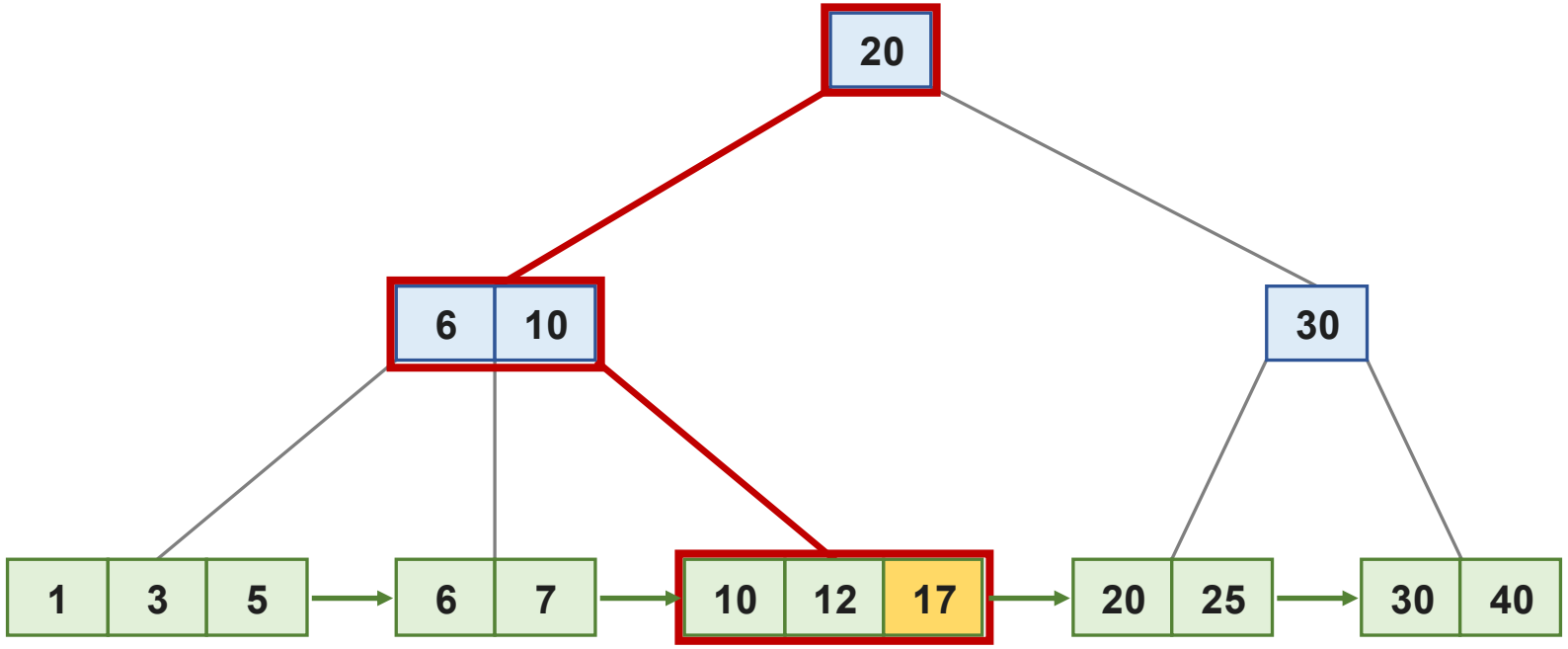
B-tree (t = 2): each key appears **once**, at any level



B+ tree (M = 4): every key is in a **leaf**; separators are copies; leaves linked

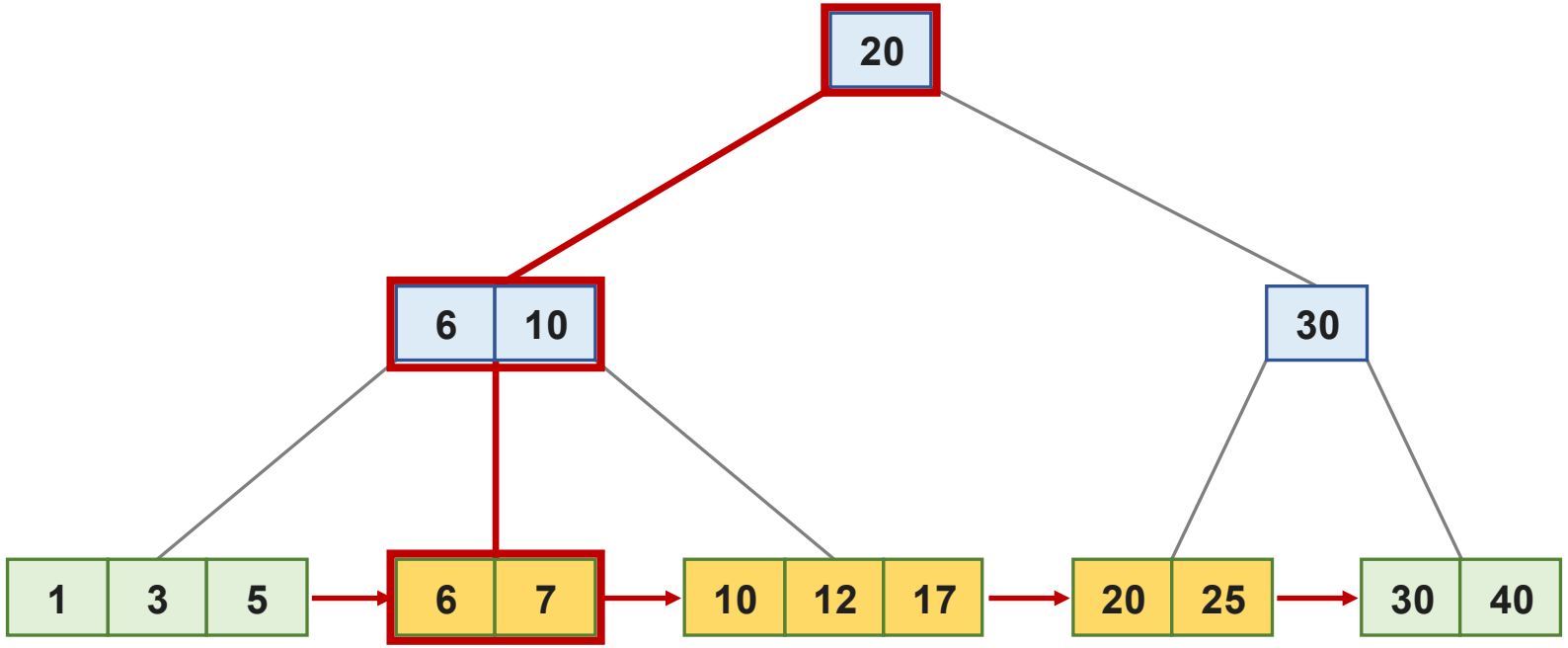


Search in a B+ Tree



- Search for **17**: at each internal node go right while key ≥ separator
- Every search ends in a **leaf** → all searches cost exactly h + 1 node reads

Range Query [6, 25]



- ① Descend **once** to the leaf that would hold 6
- ② Follow **next** pointers, output keys until a key > 25

B+ Tree in C: The Node

```c
#define M 4                          /* max children of an internal node   */
#define MAXK (M - 1)                 /* max keys in any node (here 3)      */

typedef struct BPNode {
    bool leaf;
    int  n;                          /* number of keys                     */
    int  keys[MAXK + 1];             /* +1 slot: temporary overflow        */
    struct BPNode *c[M + 1];         /* children (internal nodes only)     */
    struct BPNode *next;             /* next leaf (leaves only)            */
} BPNode;

BPNode *new_node(bool leaf) {
    BPNode *x = calloc(1, sizeof(BPNode));
    x->leaf = leaf;
    return x;
}
```

- One struct for both kinds of node; **next** is used only by leaves

B+ Tree in C: Search

```c
/* ---------------- SEARCH: always ends in a leaf ---------------- */
BPNode *find_leaf(BPNode *x, int k) {
    while (!x->leaf) {
        int i = 0;
        while (i < x->n && k >= x->keys[i])  /* keys >= separator go right */
            i++;
        x = x->c[i];
    }
    return x;
}

bool bpt_search(BPNode *root, int k) {
    BPNode *leaf = find_leaf(root, k);
    for (int i = 0; i < leaf->n; i++)
        if (leaf->keys[i] == k) return true;
    return false;
}
```

Range Query: The Code

```c
/* ---------------- RANGE QUERY [lo, hi] ---------------- */
void bpt_range(BPNode *root, int lo, int hi) {
    BPNode *leaf = find_leaf(root, lo);      /* 1. descend once          */
    while (leaf) {                           /* 2. walk the leaf chain   */
        for (int i = 0; i < leaf->n; i++) {
            if (leaf->keys[i] > hi) return;
            if (leaf->keys[i] >= lo) printf("%d ", leaf->keys[i]);
        }
        leaf = leaf->next;
    }
}
```

- Cost: O(log n) to reach the first leaf + one page per **leaf scanned**

B-Tree vs. B+ Tree

	B-tree	B+ tree
Where keys / data live	Every node	Leaves only
Internal nodes	Keys + data	Separator copies only
Duplicate keys in tree	No	Yes (separators)
Search ends at	Any level	Always a leaf
Range / sequential scan	In-order traversal	Follow leaf chain
Fan-out per page	Lower	Higher

Where Are They Used?

- **Database indexes**: B+ trees are the default index structure in relational databases
- **File systems**: B-tree / B+ tree variants index directories and file extents
- **Key-value stores** and embedded databases
- Common reason: data lives on **block storage**; one node = one block

Comer, The Ubiquitous B-Tree, ACM Computing Surveys 11(2), 1979

Summary

- Disk-based data → minimise **node visits**; use **fat nodes**
- B-tree of minimum degree t: $t - 1 \ldots 2t - 1$ keys per node; height $\leq \log_t((n+1)/2)$
- Insert: split full nodes **on the way down**; tree grows at the root
- Delete: keep $\geq t$ keys before descending; borrow or merge; tree shrinks at the root
- B+ tree: data in linked leaves → fast **range queries**; used by databases

References

- Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022, Chapter 18 (B-Trees).
- Bayer & McCreight, Organization and Maintenance of Large Ordered Indexes, Acta Informatica 1(3), 1972.
- Comer, The Ubiquitous B-Tree, ACM Computing Surveys 11(2), 1979.
- Silberschatz, Korth & Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019, Chapter 14 (Indexing).
- Code for this lecture: btree.c, bplustree.c (tested with randomised insert/delete checks).

Quiz · B-Trees · Q1

In a B-tree of minimum degree t, what is the maximum number of keys in a node?

A. t

B. $t - 1$

C. $2t - 1$

D. $2t$

Quiz · B-Trees · Q2

Why are B-tree nodes made large, with hundreds of keys each?

A. To reduce the number of key comparisons

B. So one node fills one disk page and the tree stays shallow

C. To avoid ever splitting nodes

D. So that data is stored only in the leaves

Quiz · B-Trees · Q3

How does the height of a B-tree increase?

A. Whenever a leaf splits

B. Only when the root splits

C. When a key is added to a full leaf without splitting

D. After every t insertions

Quiz · B-Trees · Q4

During insertion, when is a node split?

A. After the new key has been placed in it

B. When a full node is met on the way down

C. Only when it is a leaf

D. Only when its parent is also full

Quiz · B-Trees · Q5

Insert 8, 15, 3, 22 (in this order) into an empty B-tree with t = 2. Which of the following options can be the content of the root after these insertions?

A. [8]

B. [8, 15]

C. [22]

D. [3, 8, 15]

Quiz · B-Trees · Q6

Why are B+ trees preferred over B-trees for range queries?

A. They store fewer keys

B. All keys are in linked leaves: descend once, then follow next pointers

C. Internal nodes store the records

D. They never split nodes



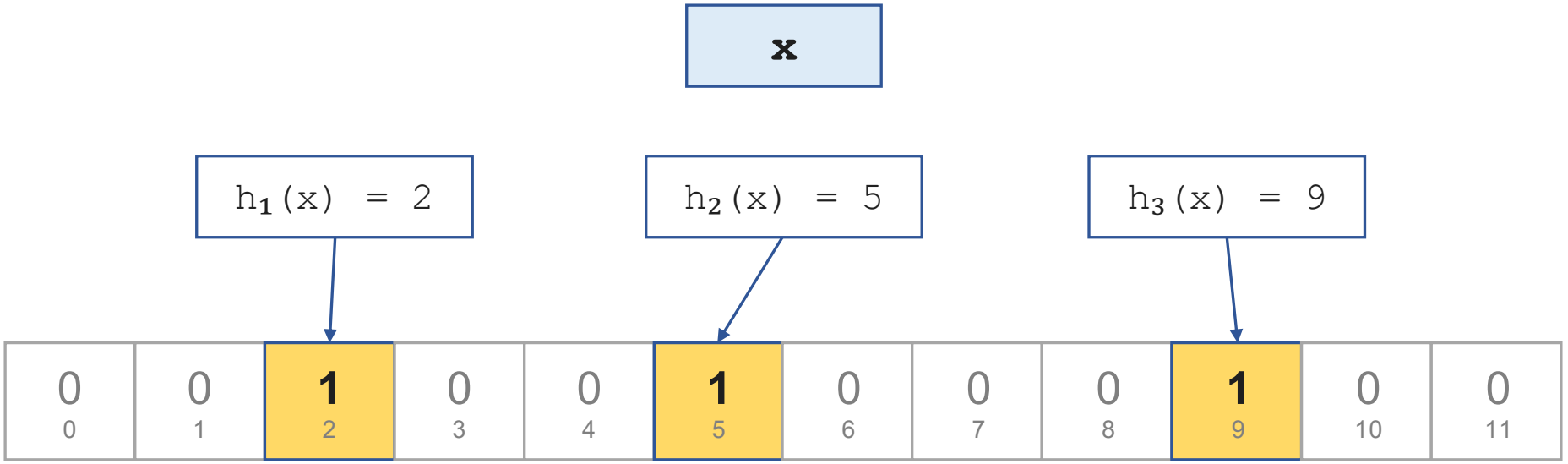
భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Bloom Filters

The Membership Problem

- Given a set S of n items, answer quickly: **is x in S?**
- Is this username taken? Is this URL on a blocklist? Is this key in this disk file?
- An exact set must store the keys: 10^9 URLs × 50 bytes ≈ **50 GB**
- A Bloom filter with a 1% error rate needs only **≈ 1.2 GB**
- Trade a small, **tunable** chance of a wrong “yes” for a large saving in space

Structure: m Bits and k Hash Functions



- Bit array B of size m (here 12), **all 0** at the start
- k hash functions (here 3) map x to indices in 0 … m − 1; set those bits to **1**
- The filter **never stores x** itself, only these bits

Code for Insert and Query

```c
/* ---------------- BLOOM FILTER ---------------- */
typedef struct {
    int m, k;                 /* bits, hash functions   */
    unsigned char *bits;       /* m bits, all 0 at start */
} Bloom;

void bf_insert(Bloom *f, const char *x) {
    for (int i = 0; i < f->k; i++)
        set_bit(f->bits, h(i, x) % f->m);    /* B[h_i(x)] = 1   */
}

bool bf_query(Bloom *f, const char *x) {
    for (int i = 0; i < f->k; i++)
        if (!get_bit(f->bits, h(i, x) % f->m))
            return false;           /* definitely not in S */
    return true;                    /* probably in S       */
}
```

Notes

- Insert: set the k bits; never checks anything
- Query: stop at the **first 0 bit**
- Both take **O(k)** time, however many items are stored
- Bits are only set, **never cleared**

Worked Example (m = 10, k = 3): Insert cat, dog

Start: all 10 bits are 0



insert(cat): hashes 1, 4, 7 → set those bits



insert(dog): hashes 3, 4, 9 → bit 4 is already 1 (shared with cat)



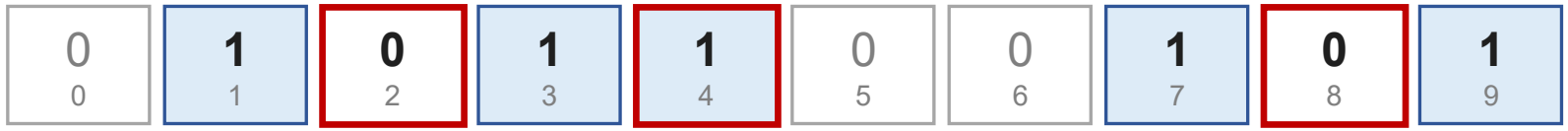
- **Yellow** = bits written by this insert; hash values are given for the example

Worked Example: Three Queries

query(cat) checks 1, 4, 7 → all 1 → **MAYBE** (correct: cat was inserted)



query(emu) checks 2, 4, 8 → bit 2 is 0 → **NO** (certain)



query(fox) checks 1, 3, 9 → all 1 → MAYBE, but fox was never inserted: **false positive**



- **Red outline** = bits the query checks; bits 1, 3 and 9 were set by cat and dog

One-Sided Error

- **No false negatives**: if x was inserted, all k of its bits are 1, and bits are never cleared
- **Some false positives**: other items may have set all k bits of a non-member
- In practice: on MAYBE, confirm with the exact (slower) store; a NO skips that work

	Filter says MAYBE	Filter says NO
x inserted	True positive	**Impossible**
x not inserted	**False positive (rate p)**	True negative

Analysis: Will a Bit Stay 0?

- Assume **ideal hashing**: each $h_i(x)$ is uniform on 0 … m − 1 and independent; fix one bit j
- One hash call misses bit j:

$$\Pr[\, h_i(x) \neq j \,] = 1 - \frac{1}{m}$$

- n inserts make kn independent hash calls; all of them must miss:

$$\Pr[\, B[j] = 0 \,] = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

- Using the limit below (large m), about a fraction e^(−kn/m) of the bits stay 0

$$\left(1 - \frac{1}{m}\right)^{m} \approx \frac{1}{e}$$

Analysis: The False-Positive Rate

- Query a non-member y. Each of its k bits is 1 with probability

$$\Pr[\, B[j] = 1 \,] \approx 1 - e^{-kn/m}$$

- A false positive needs **all k** of them to be 1:

$$p \approx \left(1 - e^{-kn/m}\right)^k$$

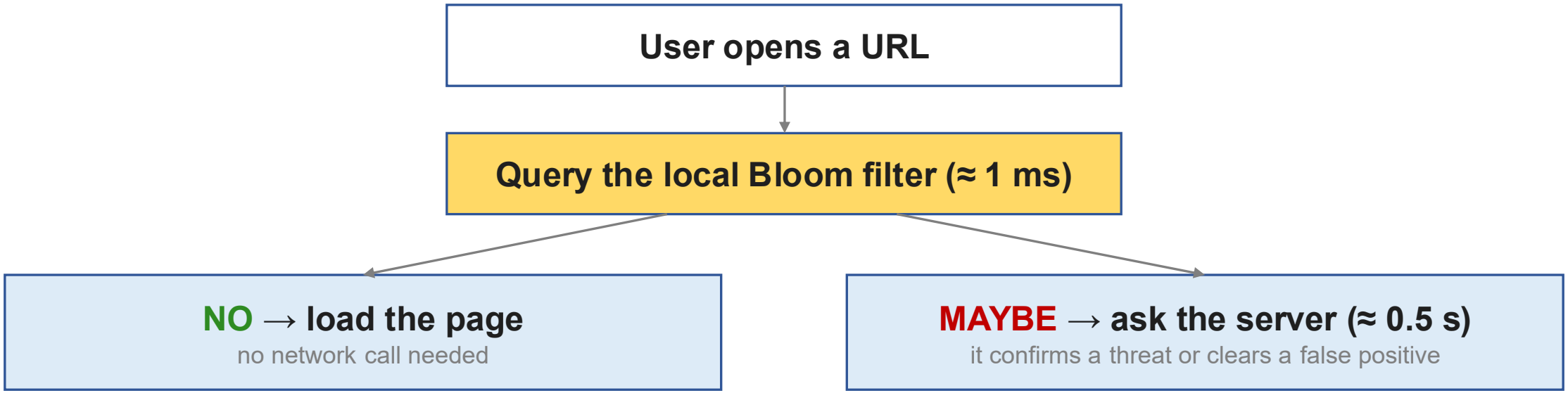
- More items n → more bits set → p **rises**; more bits m → p **falls**

Where Are They Used?

- **Web browsers**: check URLs against a local filter of malicious sites; ask the server only on MAYBE
- **Databases** (Bigtable, Cassandra, HBase, RocksDB): one filter per disk file skips reads for absent keys
- **Content delivery networks**: cache an object only on its second request (no “one-hit wonders”)
- **Networks**: routers track blocked IP addresses; replicas exchange a filter instead of a full key list
- Common pattern: a small in-memory filter in front of a slow, exact lookup

Case Study: Safe Browsing

- Early versions of Google Chrome kept a Bloom filter of known malicious URLs **inside the browser**; the full list stays on Google's servers



- Almost every URL is safe, so almost every check ends at NO without touching the network

Example adapted from UW CSE 312 (Autumn 2021), Lecture 10: Bloom filters

References: Bloom Filters

- Bloom, Space/Time Trade-offs in Hash Coding with Allowable Errors, CACM 13(7), 1970.
- Broder & Mitzenmacher, Network Applications of Bloom Filters: A Survey, Internet Mathematics 1(4), 2004.

Quiz · Bloom Filters · Q1

Which statement about Bloom filter errors is true?

A. Both false positives and false negatives are possible

B. Only false negatives are possible

C. Only false positives are possible

D. No errors occur if k is chosen optimally

Quiz · Bloom Filters · Q2

Why can't a standard Bloom filter delete an element by setting its k bits back to 0?

A. Hash functions cannot be inverted

B. A bit may be shared with other elements, so clearing it can cause false negatives

C. Deleting would take O(m) time

D. Bits can only be written once in memory

Quiz · Bloom Filters · Q3

With m and n fixed, what happens to the false-positive rate as k grows far beyond (m/n) ln 2?

A. It keeps decreasing toward 0

B. It stays constant

C. It increases, because the array fills up with 1s

D. It becomes exactly 50%

Quiz · Bloom Filters · Q4

Which of the following can a Bloom filter NOT do?

A. Answer membership queries in O(k) time

B. Use less space than storing the keys

C. List the elements it contains

D. Insert a new element without checking the others

Quiz · Bloom Filters · Q5

Two Bloom filters with the same m, k and hash functions hold sets S_1 and S_2. What is their bitwise OR?

A. A Bloom filter for $S_1 \cap S_2$

B. A Bloom filter for $S_1 \cup S_2$

C. A filter with no false positives

D. A meaningless bit array

Thank you!!



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Dr. Maunendra Sankar Desarkar

Homepage: https://iith.ac.in/~maunendra/
Twitter: https://twitter.com/msades
LinkedIn: https://www.linkedin.com/in/maunendra-sankar-desarkar-6a89907/